

Supplementary Material, Centrality-Based KV-Cache Eviction

Companion to the main report (REPORT.pdf): detailed per-press systems figures and per-task result tables. See the report §4.4–4.5 for the summary and interpretation.

S.1 Per-press systems figures

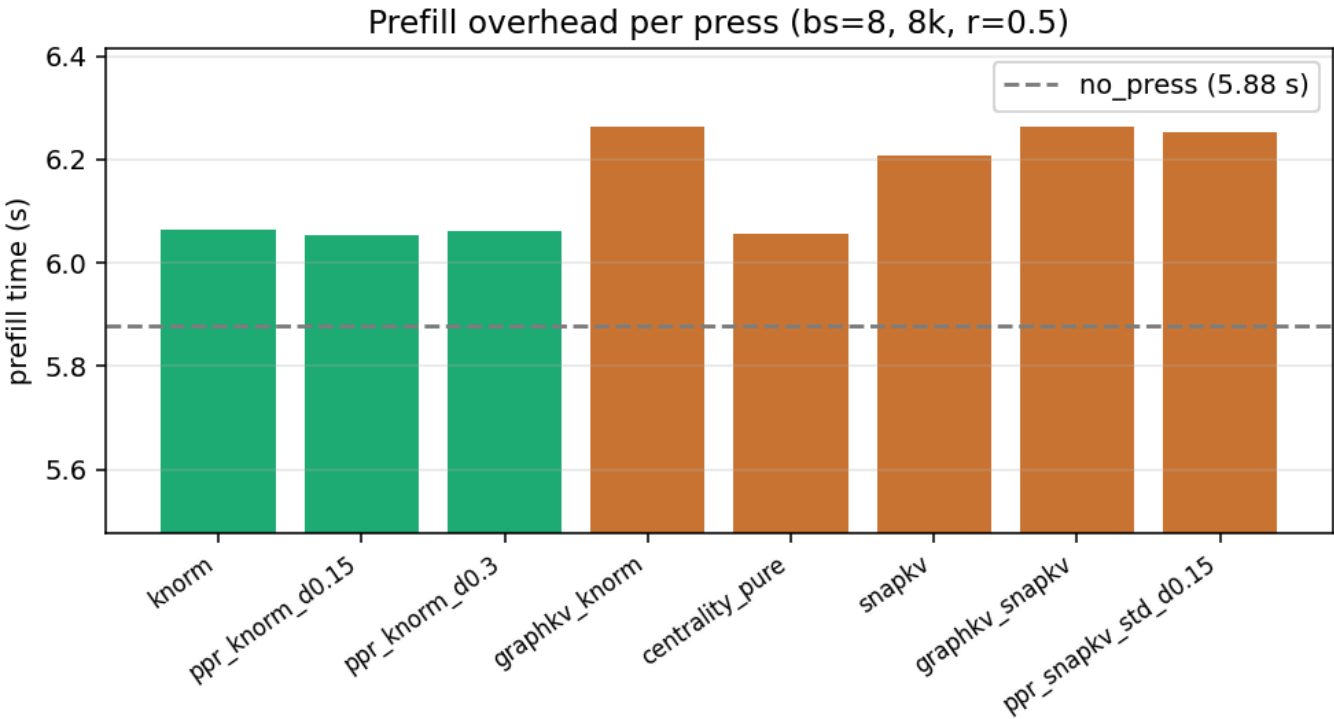


Figure S1: Prefill scoring overhead, the only press-dependent systems cost; *ppr_knorm* $\approx$ *knorm*, cheaper than *graphkv* / *SnapKV*-based presses.

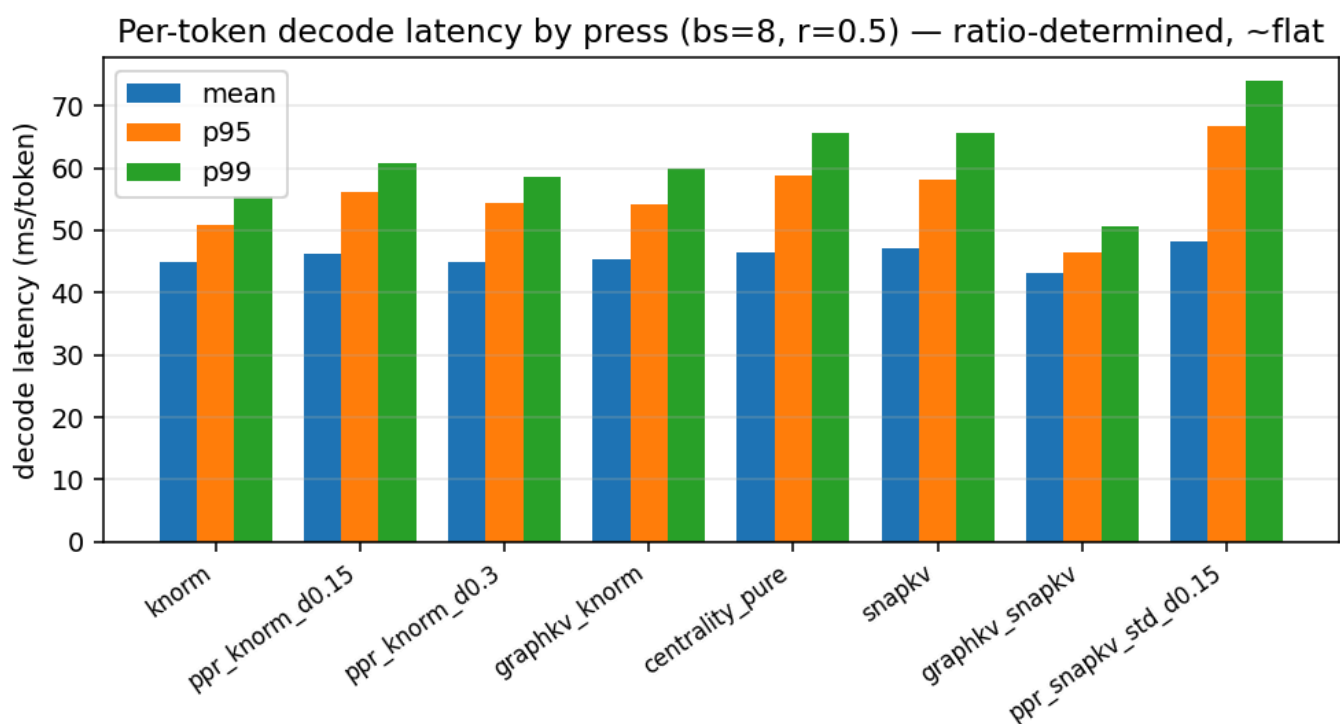


Figure S2: Per-token decode latency (mean/p95/p99) by press at $r=0.5$, ratio-determined, ~flat.

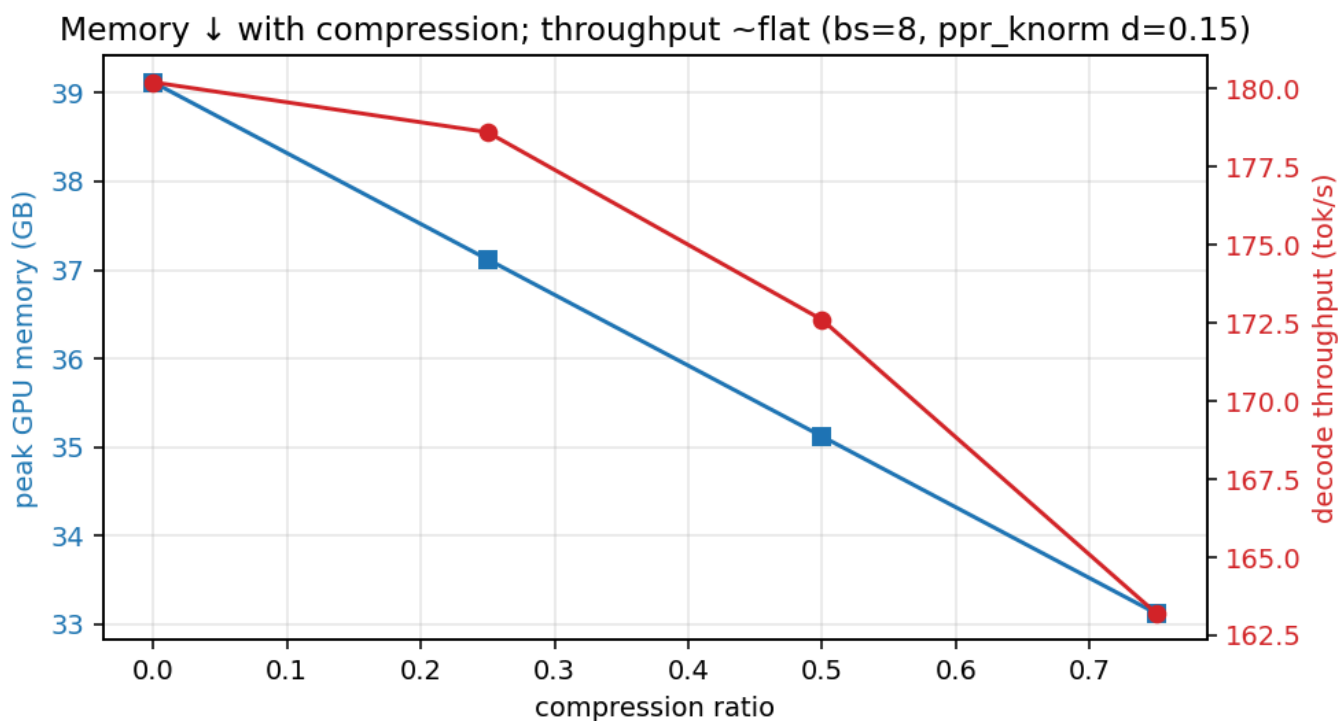


Figure S3: Peak GPU memory falls linearly with compression; decode throughput ~flat at this scale.

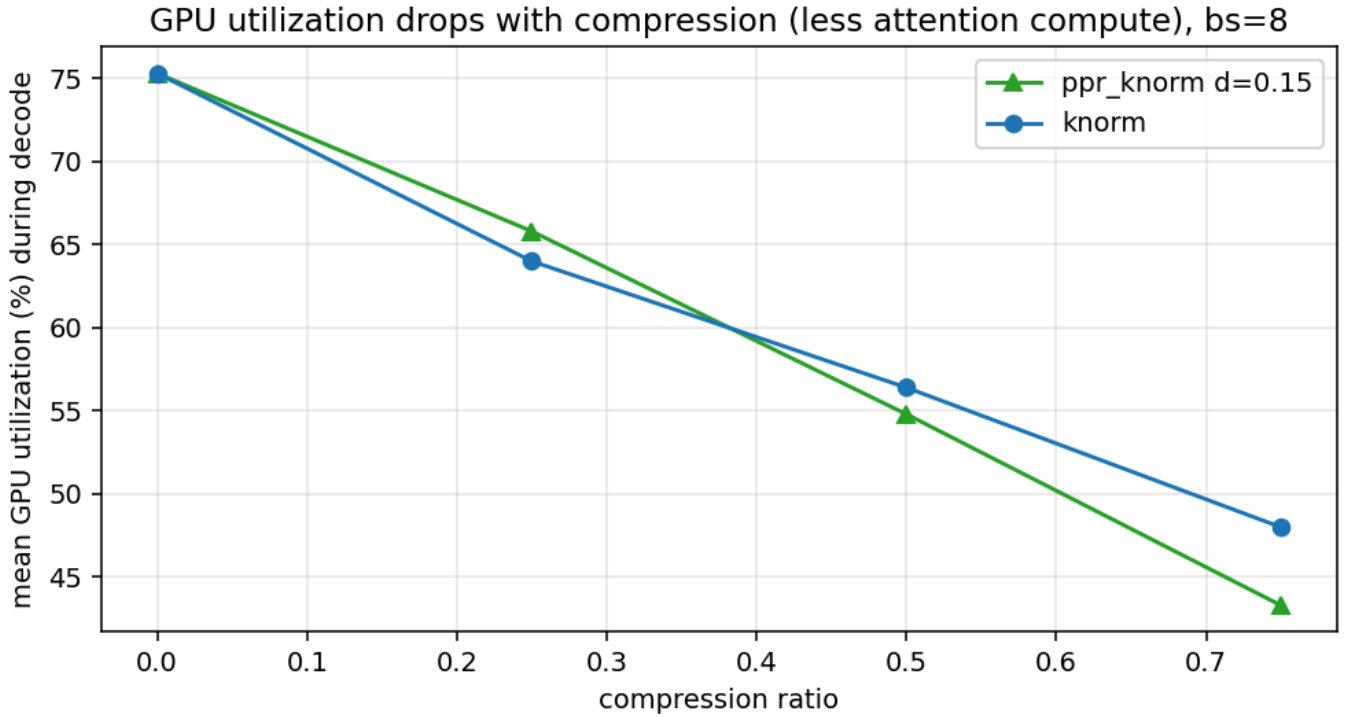


Figure S4: Mean decode GPU utilization drops with compression (less attention compute per token).

S.2 Per-task tables

RULER 4096 @ compression 0.5 (string-match %, usable tasks)

task	no_press	knorm	graphkv	snapkv	ppr d=0.15	pure
niah_single_1	100	100	100	94.7	100	0
niah_single_2	100	80.0	20.0	97.1	100	0
niah_single_3	100	2.9	5.9	0	73.5	0
niah_multikey_1	100	30.0	40.0	90.0	90.0	6.7
niah_multikey_2	100	4.8	81.0	61.9	100	0
niah_multikey_3	100	4.5	36.4	22.7	18.2	0
qa_2	58.1	32.3	48.4	48.4	48.4	29.0

LongBench multi-hop @ compression 0.5 (F1 %)

subtask	no_press	knorm	graphkv	snapkv	ppr d=0.15
hotpotqa	58.5	54.1	45.9	56.7	49.2
2wikimqa	53.1	45.9	46.7	53.5	53.4
musique	30.7	19.5	30.9	25.2	23.4

Per-run configs + metrics are under project/reproduction/results/ (config.yaml + metrics.json each; the aggregated per-example *_long.csv are report-only, regenerable, see project/reproduction/README.md). Regenerate figures with analysis/make_figures.py and paired stats with analysis/analyze_paired.py.

S.3 Iso-accuracy operating-point curve (16k, batch 8)

16k / batch 8: memory scales linearly with cache; speed moves only past the knee

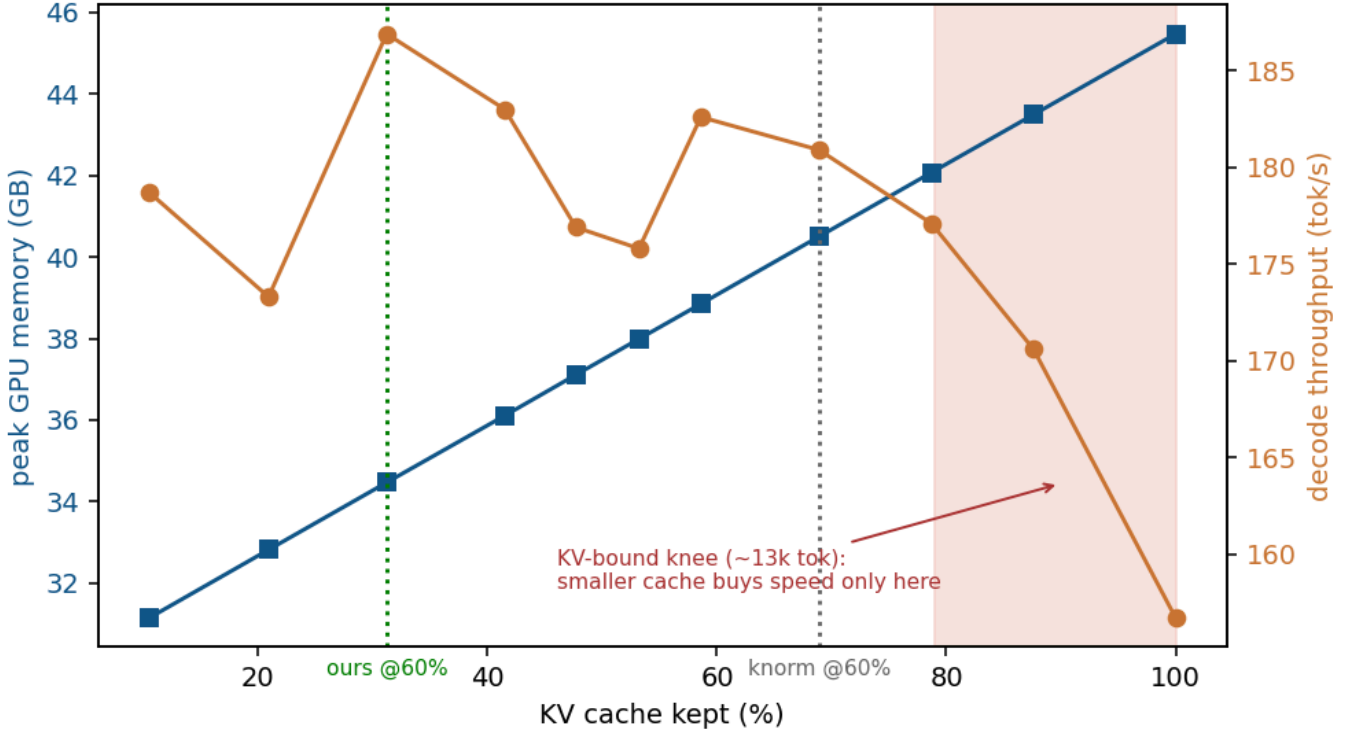


Figure S5: Directly measured peak GPU memory and decode throughput vs. kept-cache fraction (16k prompt, batch 8, `analysis/iso_systems.py` on `results/results_iso_systems_16k_bs8.csv`). Peak memory falls linearly with the cache; throughput is essentially flat once the cache is compressed (decode is weight-bound at 8B/16k/bs8). The dotted lines mark the iso-accuracy operating points for a 60 % RULER target: the base press must keep ~69 % of the cache, ours only ~31 %, same accuracy, 2.2× less cache and ~6 GB less peak memory, while throughput is unchanged (the speed win needs a more KV-bound regime).

S.4 Full leaderboard tables (Llama-3.1-8B, RULER 4k)

Complete public-board standing at submission for all 21 methods, 13-task macro `string_match`. [QA] = query-aware; **bold** = our entry; Δ vs uncompressed (95.69). Reproduce with `analysis/rank_vs_board.py`.

25% compression

#	method	macro	Δ
1	AdaKVCompactor	95.69	+0.00
2	DuoAttention	95.67	-0.02
3	DuoAttnOnTheFly	95.54	-0.15
4	ChunkKV [QA]	95.50	-0.19
5	KVzip	95.47	-0.22
6	AdaSnapKV [QA]	95.40	-0.29
7	ExpectedAttention(AdaKV-wrapped)	95.25	-0.44
8	CentralityPress (ours)	94.57	-1.12
9	SnapKV [QA]	94.52	-1.17
10	CUR	93.85	-1.84
11	Finch [QA]	93.71	-1.98
12	LagKV	92.57	-3.12
13	KeyDiff	91.59	-4.10
14	TOVA	87.35	-8.34
15	PyramidKV	82.76	-12.93
16	SnapKV	81.83	-13.86
17	StreamingLLM	79.77	-15.92
18	QFilter	79.10	-16.59
19	Knorm (our base)	76.30	-19.39
20	ObservedAttention(H2O)	74.93	-20.76
21	Random	61.94	-33.75

50% compression

#	method	macro	Δ
1	DuoAttnOnTheFly	95.66	-0.03
2	KVzip	95.43	-0.26
3	DuoAttention	95.31	-0.38
4	ChunkKV [QA]	94.76	-0.93
5	AdaKVCompactor	94.75	-0.94
6	ExpectedAttention(AdaKV-wrapped)	92.18	-3.51
7	AdaSnapKV [QA]	91.17	-4.52
8	Finch [QA]	90.30	-5.39
9	SnapKV [QA]	88.90	-6.79
10	KeyDiff	85.46	-10.23
11	LagKV	84.99	-10.70
12	CentralityPress (ours)	81.33	-14.36
13	CUR	79.41	-16.28
14	TOVA	76.21	-19.48
15	SnapKV	69.66	-26.03
16	PyramidKV	69.08	-26.61
17	QFilter	62.86	-32.83
18	StreamingLLM	59.23	-36.46
19	ObservedAttention(H2O)	54.36	-41.33
20	Knorm (our base)	52.78	-42.91
21	Random	6.88	-88.81

75% compression

#	method	macro	Δ
1	KVzip	95.24	-0.45
2	DuoAttnOnTheFly	93.71	-1.98
3	ChunkKV [QA]	90.69	-5.00
4	AdaKVCompactor	83.06	-12.63
5	AdaSnapKV [QA]	82.62	-13.07
6	SnapKV [QA]	78.76	-16.93
7	Finch [QA]	77.33	-18.36
8	ExpectedAttention(AdaKV-wrapped)	75.94	-19.75
9	DuoAttention	73.20	-22.49
10	KeyDiff	72.90	-22.79
11	LagKV	67.86	-27.83
12	TOVA	63.32	-32.37
13	CentralityPress (ours)	58.04	-37.65
14	CUR	45.57	-50.12
15	PyramidKV	43.39	-52.30
16	SnapKV	43.36	-52.33
17	QFilter	40.69	-55.00
18	StreamingLLM	37.69	-58.00
19	ObservedAttention(H2O)	34.23	-61.46
20	Knorm (our base)	30.30	-65.39
21	Random	0.56	-95.13

88% compression

#	method	macro	Δ
1	KVzip	92.80	-2.89
2	ChunkKV [QA]	72.39	-23.30
3	AdaSnapKV [QA]	71.32	-24.37
4	AdaKVCompactor	68.52	-27.17
5	SnapKV [QA]	67.96	-27.73
6	Finch [QA]	66.25	-29.44
7	KeyDiff	63.34	-32.35
8	TOVA	46.37	-49.32
9	LagKV	45.64	-50.05
10	ExpectedAttention(AdaKV-wrapped)	45.13	-50.56
11	CentralityPress (ours)	41.71	-53.98
12	DuoAttnOnTheFly	35.71	-59.98
13	QFilter	30.98	-64.71
14	SnapKV	27.21	-68.48
15	PyramidKV	27.15	-68.54
16	StreamingLLM	26.73	-68.96
17	DuoAttention	26.50	-69.19
18	ObservedAttention(H2O)	24.94	-70.75
19	Knorm (our base)	21.22	-74.47
20	CUR	19.76	-75.93
21	Random	0.18	-95.51

Appendix, Code & data artifacts

- **Press code:** `kvpress/presses/centrality_press.py`; registration in `kvpress/__init__.py`, `evaluation/evaluate_registry.py`, `tests/default_presses.py`, `README.md`; unit test in `tests/presses/test_centrality_press.py`.
- **GraphKV comparison baseline (project/additional_benchmarks/):** `decay_propagation_press.py`, `test_decay_propagation_press.py`, `run.py` (runtime-injected registry entry), `README.md`.
- **Benchmark + analysis (project/reproduction/):** `scripts/sweep_longbench.py`, `scripts/submission_grid_llama.py`, `scripts/fetch_eval_datasets.py`; `analysis/analyze_paired.py`, `analysis/make_figures.py`, `analysis/speed_memory.py`, `analysis/iso_systems.py`, `analysis/recall_probe.py`, `analysis/rank_vs_board.py`, `analysis/kvpress_leaderboard_raw.csv`.
- **Data/logs (project/reproduction/results/):** `board_grid/` (with `predictions.csv`) and `ruler_screen/` `run` directories (each with `config.yaml` + `metrics.json`); figures in `report/figures/` (`fig1-fig12`).
- **Reproducibility:** `pip install -e ".[eval]"`, `README.md` (how-to-benchmark + traceability), `scripts/benchmark.py` (one-command benchmark), `.github/workflows/centrality-ci.yml` (CI: comparison-benchmark tests on every push).